

Computer Networks — Chapter 1: Introduction

Comprehensive Study Notes (text-only preface)

Based on Tanenbaum, Feamster & Wetherall, *Computer Networks*, 6th edition (Pearson, 2021)

Table of Contents

Orientation.....	3
1.1 Uses of Computer Networks.....	4
1.1.1 Access to Information	4
1.1.2 Person-to-Person Communication	4
1.1.3 Electronic Commerce	5
1.1.4 Entertainment.....	5
1.1.5 The Internet of Things	5
1.2 Types of Computer Networks	6
1.2.1 Broadband Access Networks	6
1.2.2 Mobile and Wireless Access Networks	6
1.2.3 Content Provider Networks	7
1.2.4 Transit Networks.....	8
1.2.5 Enterprise Networks.....	8
1.3 Network Technology, from Local to Global	8
1.3.1 Personal Area Networks (PANs)	9
1.3.2 Local Area Networks (LANs).....	9
1.3.3 Home Networks	10
1.3.4 Metropolitan Area Networks (MANs)	10
1.3.5 Wide Area Networks (WANs).....	11
1.3.6 Internetworks	11
1.4 Examples of Networks	12

1.4.1 The Internet.....	12
The ARPANET	12
NSFNET.....	13
Internet Architecture	14
1.4.2 Mobile Networks.....	15
Architecture	15
Packet Switching vs. Circuit Switching.....	16
Earlier Generations: 1G, 2G, 3G	16
Modern Generations: 4G and 5G.....	16
1.4.3 Wireless Networks (WiFi)	17
1.5 Network Protocols	18
1.5.1 Design Goals	18
1.5.2 Protocol Layering	19
1.5.3 Connections and Reliability.....	20
1.5.4 Service Primitives.....	21
1.5.5 The Relationship of Services to Protocols	22
1.6 Reference Models	22
1.6.1 The OSI Reference Model	22
1.6.2 The TCP/IP Reference Model.....	23
1.6.3 A Critique of the OSI Model and Protocols	24
1.6.4 A Critique of the TCP/IP Reference Model	25
1.6.5 The Model Used in This Book	25
1.7 Standardization	26
1.7.1 Standardization and Open Source.....	26
1.7.2 Who's Who in the Telecommunications World	26
1.7.3 Who's Who in the International Standards World	27
1.7.4 Who's Who in the Internet Standards World	28

1.8 Policy, Legal, and Social Issues	28
1.8.1 Online Speech.....	29
1.8.2 Net Neutrality	29
1.8.3 Security.....	29
1.8.4 Privacy	30
1.8.5 Disinformation.....	30
1.9 Metric Units.....	30
1.10 Outline of the Rest of the Book	31
1.11 Summary	31

Orientation

This document is a complete, detail-preserving set of notes for **Chapter 1, “Introduction”** of Tanenbaum, Feamster and Wetherall’s *Computer Networks* (6th ed.). It is written as a readable preface: you can skim it front-to-back to build the full mental map of the chapter before studying the book itself. Figures are not reproduced, but wherever a figure carries information (a table, a comparison, a sequence), that information is written out in words or as a table so that nothing is lost.

The chapter’s one-sentence thesis: three centuries each had a dominant technology (the 18th century mechanical systems of the Industrial Revolution, the 19th century the steam engine, the 20th century information gathering/processing/distribution), and these are now converging. A **computer network** is defined as *a collection of interconnected, autonomous computing devices*. “Interconnected” means they can exchange information, over copper wire, fiber optics, microwaves, infrared, or communication satellites; “autonomous” distinguishes a network from a single controlled system. Networks come in all sizes, and networks can themselves be connected together — the **Internet** is the best-known example of a network of networks. The chapter surveys the uses of networks, the hardware and scale categories, concrete example networks, the protocol and layering concepts, the reference models (OSI and TCP/IP), standardization, and the policy/legal/social issues, closing with metric units and a roadmap of the book.

A recurring distinction worth holding onto from the start: the **uses** (applications) of a network are different from the **network itself** (the underlying hardware and software), just as a program is different from the computer it runs on.

1.1 Uses of Computer Networks

Networks are used for access to information, person-to-person communication, electronic commerce, entertainment, and connecting physical “things.” The authors stress that pinning down applications is hard because people find surprising new uses, but the categories below organize the main ones.

1.1.1 Access to Information

Access to information today takes the form of browsing the **World Wide Web** for information on the arts, business, cooking, government, health, history, hobbies, recreation, science, sports, travel, and more — plus consuming news, which increasingly comes from online sources that can be personalized and curated. A downside noted here is that personalization relies on tracking individual behavior, which can be misused, and advertising supported by this data (behavioral/targeted advertising) raises concerns about data collection.

Digital libraries and online repositories are a major use: professional organizations such as the **ACM** and the **IEEE Computer Society** already put their journals and conference proceedings online, and e-book readers and online libraries have changed reading. Much of this is organized as the **client-server model**: a **client** process on a user’s machine sends a request over the network to a **server** process, which does the work and sends back a reply. The server commonly handles requests from a large number (hundreds or thousands) of clients at once. The **Web** is the most popular realization of the client-server model — a page request goes to a Web server which returns the page.

An alternative structure is **peer-to-peer (P2P)** communication, in which individuals form a loose group with no fixed division into clients and servers: every person can communicate with others, and there is no central database. Example applications include **BitTorrent** (file sharing with no central database of content; introduced conceptually with Cohen’s 2003 design) and the historical **Napster** (music sharing, shut down around 2000 in the largest copyright-infringement case in history to that point). P2P has many legitimate uses too. Even ordinary email can be viewed as peer-to-peer at the human level. The distinction between an application being a legitimate or illegitimate use is independent of the technology.

1.1.2 Person-to-Person Communication

Email is the flagship: used daily by huge numbers of people, it carries audio, video, text, and pictures. **Instant messaging / real-time text**, descended from the UNIX `talk` program (circa 1970), lets two people type at each other in real time; **Twitter**-style microblogging lets people send short “tweets” to their followers. Networks support audio and video streaming (Internet radio, music and video services), and enable **distance learning** — attending classes remotely — and telemedicine, which will only grow.

Social networks such as Facebook are built on person-to-person links: users maintain a profile and a set of “friends,” and information flows along the resulting graph (friends,

friends-of-friends). A **wiki** is a collaborative website whose content members collectively edit; **Wikipedia** is the best-known, but wikis are used internally in many organizations too. These communications tools create online communities and can both strengthen and strain social ties.

1.1.3 Electronic Commerce

Online shopping is mainstream: consumers browse electronic catalogs of thousands of companies and have goods delivered. Financial institutions let customers manage accounts and investments electronically (and “fintech” extends this). Online **auctions** of second-hand goods are, in structure, consumer-to-consumer but run through a central server that lists goods and manages bidding. The chapter classifies e-commerce by the parties involved, which can be remembered as a small table of forms and examples:

Tag	Full name	Example
B2C	Business-to-consumer	Ordering books from an online retailer
B2B	Business-to-business	Car manufacturer ordering tires from a supplier
G2C	Government-to-consumer	Government distributing tax forms electronically
C2C	Consumer-to-consumer	Auctioning second-hand goods online
P2P	Peer-to-peer	File sharing; music/video sharing

1.1.4 Entertainment

Entertainment is a huge and growing use. Music, radio, television, and movies are distributed over the Internet (digital formats such as MP3 for audio and high-definition video); **IPTV** (IP television) and **media/video streaming** let people watch what they want when they want. Content can be moved around the home wirelessly between devices. Users can search for any movie ever made and have it delivered instantly. **Interactive** film and television, where the viewer influences the plot, is an emerging form. **Game playing** is enormous: multi-person real-time simulation games, flight simulators, and shared **virtual worlds** rendered in 3D, in which large numbers of users interact simultaneously. (Streaming video is, in fact, the dominant category of Internet traffic by volume.)

1.1.5 The Internet of Things

The vision of **ubiquitous computing** (Mark Weiser, 1991) is of computing embedded invisibly into everyday life. The **Internet of Things (IoT)** realizes this: information-gathering and -acting devices are networked and reachable. Examples include home security systems with sensors and cameras; **smart meters** for electricity, gas, and water that report usage to the utility; smoke/CO detectors that can call the fire department; smart refrigerators; and consumer electronics that are networked by default. A memorable

illustration: a sensor can report how much water each shower uses. Power-line networking (using existing electrical wiring to carry data) is one way such devices connect. IoT raises the device count on networks dramatically and, as later sections note, creates new security and privacy exposure.

1.2 Types of Computer Networks

This section classifies networks by the *role they play* rather than their size. The main categories are **access networks** (which connect users and IoT devices to the rest of the network, over broadband wired or mobile/wireless links), **content-provider networks** (data centers and distribution networks that host and serve content), **transit networks** (which carry traffic between other networks), and **enterprise networks** (which connect the computers of a single organization).

1.2.1 Broadband Access Networks

The authors open with Ken Olsen’s famously wrong 1977 remark (the founder and president of **Digital Equipment Corporation, DEC**): “There is no reason for any individual to have a computer in his home.” People did want home computers — initially for word processing and games, but the decisive reason became **access to information, communication, and shared content**, i.e., Internet access. Embedded computers (in appliances and IoT devices) add still more demand.

The value of connectivity is captured by **Metcalf’s law** (named for Robert Metcalfe, inventor of Ethernet; popularized by George Gilder, 1993): the value of a network is proportional to the *square* of the number of users, because the number of potential connections grows as the square of the number of participants. This is why networks become dramatically more useful as they grow.

Broadband access means high-capacity connections (over copper, coaxial cable, or fiber) delivering megabit-to-gigabit rates to homes and businesses. In developed regions, wired broadband (cable, DSL, fiber) is common; in many developing regions, mobile/wireless access predominates because wired infrastructure is sparse.

1.2.2 Mobile and Wireless Access Networks

Mobile computers — laptops, tablets, and above all smartphones — are the fastest-growing segment and have overtaken desktop computers. Wireless matters wherever wires cannot go (cars, boats, airplanes) and wherever mobility is needed. Cellular networks provide wide-area wireless; **802.11 (“WiFi”) hotspots** provide local wireless. Fleets of trucks, taxis, delivery vehicles, and field-service technicians gain value from being connected; dispatchers can reach drivers (classic taxi dispatch), and the “sharing economy” (ride-hailing services such as Uber and Lyft) is built on mobile connectivity. The military relies on wireless mobility in the field.

A key conceptual point: **wireless and mobile are not the same thing**. The chapter lays out the four combinations:

	Wireless	Mobile	Typical application
Fixed, wired	No	No	Desktop computer in an office
Wireless, not moving	Yes	No	A network in an unwired old building
Wired, but portable	No	Yes	A laptop used in hotel rooms (plugged in at each)
Truly mobile and wireless	Yes	Yes	Store inventory with a handheld; a smartphone in a moving car

So a device can be wireless but stationary (a desktop on WiFi), or mobile but not continuously wireless (a laptop carried between wired locations), with the most demanding case being devices that are both. The mobile phone is the great driver; smartphones (iPhone, Galaxy, etc.) combine phone and computer. **SMS (text messaging)** was long enormously profitable for carriers and is now largely displaced by over-the-top messaging apps (WhatsApp, Signal, Messenger). Other mobile devices include e-book readers and digital picture frames.

Location is central to mobile services. Devices locate themselves with **GPS** and, indoors or as a supplement, via **WiFi triangulation**. This enables **location-dependent services** (find the nearest restaurant, local maps and ads) and **geo-tagging** of photos and posts. **M-commerce** (mobile commerce; Senn 2000) lets phones authorize payments; **NFC (Near Field Communication)** and RFID-based smartcards let a phone act as a payment or transit card. Mobile payment saves stores transaction fees but also lets consumers price-check competitors in the aisle; consumers are already accustomed to paying through their phones.

Mobility also powers **sensor networks**: cars report diagnostics and road conditions (e.g., detecting potholes — Hull et al., 2006), wildlife is tracked (zebra tracking — Juang et al., 2002), and research has pushed toward tiny sensors (a cubic-millimeter-scale computer — Warneke et al., 2001). Wireless parking meters and similar embedded wireless devices form a multi-billion-dollar market (Harte et al. cite figures on the order of \$10 billion).

1.2.3 Content Provider Networks

Much Internet content is served from **data centers** housing hundreds of thousands to millions of servers, sometimes in buildings over a kilometer long. This underlies **cloud computing**: organizations rent computing and storage rather than owning it. Inside a data center, the key performance measure is **cross-section bandwidth** (the bandwidth between arbitrary pairs of servers). The old **tree topology** (servers → access switches → aggregation switches → core switches) did not scale, so modern data-center networks use richer topologies to provide high bisection bandwidth.

To deliver content to users efficiently, providers build **CDNs (Content Delivery/Distribution Networks)**: large providers such as Google, Facebook, and Netflix operate their own CDNs, while others pay hosting/CDN companies such as **Akamai** and **Cloudflare**. A CDN places copies (**replicas**) of content near users and performs **replica selection**, choosing which copy to serve based on distance, server load, and network congestion so that users get fast responses.

1.2.4 Transit Networks

The Internet is built from many independently operated networks. An organization that produces content (a content host) must get it to the access ISPs that reach users. **Transit networks** (also called **backbone networks**) carry traffic from one network to another and typically charge both the sender's and the receiver's networks — historically a profitable business. Networks can also connect **directly** to one another to avoid paying a transit provider. Two long-running trends are reshaping this: **consolidation of content** into a few very large providers, and **expansion of access-ISP footprints** (access ISPs growing geographically). As a result, large networks increasingly interconnect directly and rely on transit networks less — often keeping transit only as a backup path.

1.2.5 Enterprise Networks

Organizations with more than one computer network them for two main reasons. **Resource sharing** makes programs, equipment, and especially data available to anyone regardless of physical location — the classic small example is many employees sharing one printer; the deeper point is sharing information. **Reliability/currency of information** matters: in banking a five-minute-old balance may be fine, but in some manufacturing contexts data must be current to within seconds.

Enterprises span multiple sites, so they use **VPNs (Virtual Private Networks)** to join geographically separated offices into one logical network, overcoming the “tyranny of geography.” Within the enterprise, the client-server model again dominates (employee client machines talking to company servers). Networks also serve **communication**: email replaced much internal paper mail; **VoIP / IP telephony** carries voice over the data network; video conferencing, desktop sharing, and telemedicine add real-time collaboration. The authors frame a race between communication and transportation — networking lets work move to people instead of people moving to work. Finally, enterprises do business electronically with other companies, especially suppliers and customers (e-commerce at the B2B level).

1.3 Network Technology, from Local to Global

Having covered roles, the chapter now classifies networks by **scale**, from smallest to largest: personal area networks, local area networks (including home networks), metropolitan area networks, wide area networks, and internetworks. Scale is a good classifier because networks of different sizes use different technology.

1.3.1 Personal Area Networks (PANs)

A **PAN** spans the range of a single person — roughly a few meters. The canonical use is connecting a computer or phone to its peripherals (mouse, keyboard, headphones, printer) and to wearables (a smartwatch or fitness band talking to a smartphone). The dominant PAN technology is **Bluetooth**, which uses a **master-slave** design: the master device (e.g., the phone) tells the slave devices (e.g., the headset) what addresses to use, when they may transmit, how long they may transmit, what frequencies to use, and so on. Bluetooth is covered in detail in Chapter 4.

1.3.2 Local Area Networks (LANs)

A **LAN** is a privately owned network within a single building or campus. LANs are everywhere (homes, offices, factories). There are two broad kinds.

Wireless LANs let each computer use a radio modem and antenna to communicate. Very often devices talk to an **Access Point (AP)**, also called a **wireless router** or **base station**, which relays packets between wireless devices and (usually) to the wired network and the Internet. If stations relay for one another, the result is a **mesh network**. Wireless LANs are convenient (no cabling), important in developing regions and in large homes. The standard is **IEEE 802.11**, universally nicknamed **WiFi**, with speeds historically ranging from 11 Mbps (802.11b) up to about 7 Gbps (802.11ad).

This is where the book pins down its **speed/unit convention**: line (data) rates are quoted in decimal units — 1 Mbps = 1,000,000 bits/second, 1 Gbps = 1,000,000,000 bits/second — while powers of two (1 KB = 2^{10} bytes, 1 MB = 2^{20} bytes) are reserved for memory/storage sizes. (The full table appears in §1.9.)

Wired LANs run over copper, coaxial cable, or increasingly fiber. Because a LAN is restricted in size, the worst-case transmission time is bounded and known in advance, which simplifies design and improves performance; speeds run roughly from 100 Mbps to 40 Gbps with low latency (delay) and very few errors. The dominant wired LAN is **IEEE 802.3**, universally known as **Ethernet**. Modern Ethernet uses a **switch**: a device with **ports**, each connecting to one computer by a point-to-point link; the switch forwards frames only to the port that needs them. Switches can be interconnected, and an anti-looping algorithm (the spanning-tree approach of **Radia Perlman, 1985**) prevents frames from circulating forever when there are loops. A single physical switched network can be partitioned into **VLANs (Virtual LANs)** by tagging ports/traffic with “colors” (e.g., green for one department, red for another), so that logically separate LANs share physical hardware.

Historically, **classic Ethernet** worked differently: all machines shared a single cable and transmissions were **broadcast**; because many machines could want to send at once, the network used **distributed arbitration** — if two frames **collided**, each sender waited a random time and retried. This introduces the general problem of **channel allocation**. Allocation can be **static** (e.g., dividing time into round-robin slots, which wastes capacity when a station has nothing to send) or **dynamic** (on demand). Dynamic allocation can in

turn be **centralized** (an arbiter — e.g., a cable headend — decides who goes next) or **decentralized** (each station decides for itself, as in classic Ethernet).

1.3.3 Home Networks

A home network is a special LAN with distinctive requirements. Its devices are highly heterogeneous — computers, phones, tablets, TVs and media players, game consoles, cameras, printers, thermostats, lights, locks, appliances, and other IoT gadgets — and are operated by non-experts. From this the authors derive four properties that make home networks hard:

- They must be **easy to install and maintain**. A major real-world failure mode is wireless routers being returned because customers cannot get them working; the network has to work for people who are not technically sophisticated.
- They demand **high security and reliability**, because the stakes are personal: a flaw could let a burglar disarm an alarm or unlock a door. Even *encrypted* traffic can leak information — an eavesdropper may infer which devices are present and when the home is occupied from the *pattern, timing, and volume* of traffic, without reading its contents.
- They **evolve organically** and must interoperate across vendors and generations: a voice assistant should control lights from a different manufacturer, and devices of different ages (and different versions of 802.11, USB, etc.) must coexist. The authors hope to avoid “interface wars” like those of the past.
- They face **low profit margins** on cheap mass-market devices, so the technology must be inexpensive. Wireless is attractive precisely because it avoids the cost and disruption of running cables.

A practical note: wireless is usually the performance bottleneck, and when it is slow, users tend to blame the ISP. The 2.4 GHz band penetrates walls better than 5 GHz. Security weaknesses such as the early WEP scheme are a particular concern in the home. **Power-line networks**, which send data over the home’s electrical wiring (on different frequency bands than the power itself), are one wired option that needs no new cables.

1.3.4 Metropolitan Area Networks (MANs)

A **MAN** covers a city. The archetype is the **cable television** network, which grew from **community antenna** systems (a big shared antenna piping broadcast TV to homes that could not receive it well). As cable operators sought new business, they used unused spectrum on the coaxial cable to offer two-way Internet access; the equipment at the operator end is the **head-end / CMTS (Cable Modem Termination System)**. Other MAN technologies include **WiMAX (IEEE 802.16)**, which never really caught on, and cellular broadband such as **LTE** and **5G**, which do serve metropolitan-scale wireless access. Cable is revisited in Chapter 2.

1.3.5 Wide Area Networks (WANs)

A **WAN** spans a large geographic area — a country, a continent, or several. A WAN may be a private **enterprise WAN** (connecting one company's sites) or a commercial **transit/ISP network**. In the classic model, the machines that run user programs are **hosts**; the hosts are connected by a **communication subnet** (or just **subnet**) owned and operated by a telephone company or ISP. The subnet has two components: **transmission lines** (which move bits — copper, coax, fiber, or radio) and **switching elements / routers** (specialized computers that connect three or more lines and decide where to forward incoming packets).

Two algorithms matter here, and the distinction recurs throughout the book: the **routing algorithm** decides what paths packets should take (the global, slower decision), while the **forwarding algorithm** is the per-packet action a router takes when a packet arrives (look it up, send it out the right line). (Routing is the subject of Chapter 5.) Note that “subnet” has a *second* meaning later in the book, related to addressing — context distinguishes them.

Several distinctions separate WANs from LANs: in a WAN, the **hosts and the subnet are often owned by different parties** (your company owns the hosts; a carrier owns the subnet), and keeping them separate simplifies the design. A WAN's routers typically connect networks that use **different technologies** (e.g., an Ethernet at a site and a **SONET** optical link in the carrier core), which already makes a WAN a kind of internetwork. And what is connected may be individual computers *or* entire LANs.

Rather than lease dedicated lines, a company can build a WAN using the Internet itself: a **VPN (Virtual Private Network)** creates **virtual links** — logical connections built on top of the public Internet — to join sites. The advantage is flexibility (reuse the Internet connectivity a site already has); the disadvantage is that the company no longer has dedicated control over those links and must share the underlying capacity. When the subnet is run as a business by an ISP, it is a transit/ISP network, and the customers are the subscribers. **Satellite** systems are another WAN style and are naturally **broadcast** (anything the satellite transmits can be received by all stations below it).

Cellular networks are wide-area wireless and have gone through **five generations**: **1G** carried analog voice; **2G** digital voice; **3G** digital voice *and* data; **4G** was purely digital, carrying even voice as data; and **5G** is pure digital with much higher speeds and lower delay. Cellular base stations cover areas measured in kilometers (versus tens of meters for a WiFi AP), with a wired backbone behind them; cellular speeds (on the order of 100 Mbps) are typically below top WiFi rates (up to ~7 Gbps). Modern enterprise WANs often use **SD-WAN (Software-Defined WAN)**, which mixes leased lines with cheaper commodity Internet links and uses software control to meet a target **SLA (Service Level Agreement)**; this is an application of **SDN (Software-Defined Networking)**.

1.3.6 Internetworks

An **internetwork** (or lowercase **internet**) is a collection of interconnected networks — created when distinct, independently operated, and often technologically incompatible networks are joined. The authors are careful with vocabulary: a **network** here means the

combination of a subnet plus its hosts (though “network” is used loosely in practice); an **internet(work)** is the *interconnection of distinct networks*; and the **Internet** (capital I) is one specific, global internetwork. Connecting a LAN and a WAN, or two LANs, generally yields an internetwork, because they differ fundamentally in technology or are run by different parties who must *pay* to interconnect.

The device that joins two networks is a **gateway**, and gateways are classified by the protocol layer at which they operate. The most important case is a gateway operating at the **network layer**, which is exactly a **router**: a router is a network-layer gateway that forwards packets between different networks.

1.4 Examples of Networks

This section grounds the abstractions in three concrete, important networks: the Internet, mobile (cellular) networks, and wireless (WiFi) networks.

1.4.1 The Internet

The Internet is not a single network but a vast collection of different networks that use common **protocols** and provide common **services**. It is unusual in that no single organization planned it or controls it. To understand it, the authors trace its history (and recommend Naughton 2000, McCullough 2018, and Severance 2015 for fuller accounts).

The ARPANET

The story begins in the late 1950s at the height of the **Cold War**. The U.S. Department of Defense wanted a command-and-control network that could survive a nuclear war. The existing **telephone network** was far too vulnerable: it was organized as a hierarchy of switching offices (toll offices), and destroying a few key switches would fragment it into isolated islands.

Around 1960, **Paul Baran** at the **RAND Corporation** proposed a radically different design: a **distributed, highly fault-tolerant** network using **digital packet-switching**. Baran documented it in a series of reports (notably 1964). **AT&T**, wedded to analog circuit switching, dismissed the idea — reportedly refusing even to let anyone else build it — and the proposal died.

Meanwhile, the launch of the Soviet **Sputnik** satellite in **October 1957** shocked the U.S. and prompted President Eisenhower to create **ARPA (the Advanced Research Projects Agency)** within the DoD. ARPA had no scientists or labs of its own; it worked by issuing research grants and contracts to universities and companies. After several false starts, ARPA’s program manager **Larry Roberts** turned, in 1967, to networking. A key idea came from **Wesley Clark**, who suggested building a **packet-switched subnet** in which each host would be attached to its own dedicated router. Roberts presented the plan at an ACM SIGOPS symposium in **Gatlinburg** in 1967, where he learned of related work by **Donald Davies** at the **NPL (National Physical Laboratory)** in England — which had itself cited Baran.

ARPA's design used routers called **IMPs (Interface Message Processors)** connected by **56-kbps** leased telephone lines, with each IMP connected to at least two others so that traffic could be **rerouted** around failures. Each packet carried a full destination address. A host connected to its local IMP by a short wire; a host could hand the IMP a message of up to **8063 bits**, which the IMP broke into **packets of at most 1008 bits** and forwarded independently — making the ARPANET **the first store-and-forward packet-switching network**. ARPA put the subnet out to tender; of twelve bids, it selected **BBN (Bolt, Beranek and Newman)** in December 1968. BBN used minicomputers — **Honeywell DDP-316** machines with 12K 16-bit words of memory and no disks — as the IMPs. The subnet software was split into the IMP end of the host-IMP connection, IMP-to-IMP protocols, and a source-IMP-to-destination-IMP protocol to improve reliability.

Outside the subnet, host software was also needed. BBN initially felt its job ended once it correctly received and delivered messages; Roberts pointed out that the hosts needed to do more. At a summer 1969 meeting at **Snowbird, Utah**, graduate students worked out host software with no grand master plan. The network went live in **December 1969** with four nodes: **UCLA, UCSB (Santa Barbara), SRI, and the University of Utah** — chosen partly because they had ARPA contracts and highly incompatible, very different host computers, which made the network more useful as a demonstration. (The very first host-to-host message had actually been sent about two months earlier, from **Leonard Kleinrock's** group at UCLA toward SRI, famously crashing after two characters.) The network grew quickly through the early 1970s.

As ARPA added **satellite and packet-radio** networks, the existing protocols proved unable to internetwork across these dissimilar networks. A famous demonstration had a van drive around the San Francisco area using packet radio to SRI, through the ARPANET, and over a satellite link to **London**. This problem motivated the invention of **TCP/IP** by **Vint Cerf and Bob Kahn (1974)**, explicitly designed to interconnect multiple networks ("internetworking"). ARPA funded implementations on major platforms, and crucially the protocols were built into **Berkeley UNIX** with the **socket** programming interface in **4.2BSD**, which — together with the spread of VAX minicomputers and LANs — made TCP/IP spread widely from the mid-1970s onward.

NSFNET

By the late 1970s the **NSF (National Science Foundation)** saw that the ARPANET's requirement of a DoD contract excluded most universities. NSF first funded **CSNET (1981)**, reachable by dial-up and leased lines. In the late 1980s NSF built a successor **open to all university research groups**: a **backbone** connecting six NSF **supercomputer centers** (San Diego, Boulder, Champaign, Pittsburgh, Ithaca, and Princeton), with each center getting a **fuzzball** (an LSI-11-based microcomputer router) and the centers linked by **56-kbps** lines. This was the first **TCP/IP WAN**. NSF also funded roughly 20 **regional networks** that connected to the backbone, and the whole thing was called **NSFNET**; it connected to the ARPANET through a link between an IMP and a fuzzball at **Carnegie-Mellon** (around 1988).

NSFNET was an instant success and immediately overloaded. NSF arranged an upgrade through a consortium, **MERIT** (with **MCI** and **IBM**): fiber links at **448 kbps** (MCI; later part of Verizon in 2006), with IBM PC-RT machines as routers, upgraded to **1.5 Mbps** by 1990. A nonprofit, **ANS (Advanced Networks and Services)**, was formed by MERIT, MCI, and IBM; around 1990 ANS took over and upgraded the backbone to **45 Mbps (ANSNET)**, later selling the network to America Online. Commercial IP services multiplied, and to interconnect them NSF funded four **NAPs (Network Access Points)** operated by **PacBell (San Francisco)**, **Ameritech (Chicago)**, **MFS (Washington, D.C.)**, and **Sprint (New York City / Pennsauken, NJ)**, where competing networks could exchange traffic. In this way the infrastructure the DoD and NSF had created was handed off to industry.

Through the 1990s other countries built their own backbones (e.g., **EuropaNET** and **EBONE** in Europe, moving from 2 Mbps to 34 Mbps). The **World Wide Web**, invented by Tim Berners-Lee at CERN and exploding in the early 1990s (the first WWW conference was held at **CERN in 1994**), drove explosive growth; the Internet Systems Consortium counted well over **600 million** visible hosts. The *usage* also evolved in waves: first email among academics plus newsgroups, remote login, and file transfer; then email for everyone; then the Web; then peer-to-peer file sharing (Napster); and then real-time media and social media — with **streaming video** (Netflix, YouTube) eventually becoming the dominant traffic type.

Internet Architecture

The modern architecture is shaped by **convergence** (voice, video, and data over one network — the “triple play” of a single provider offering all three). The path from a home follows this chain. A home computer or device attaches to a home router, which reaches an **access ISP** over the **last mile**. Common last-mile technologies, by increasing capability and cost, include:

- **Cable: HFC (Hybrid Fiber-Coaxial)** plant, using **DOCSIS**-standard **cable modems** talking to a **CMTS** at the head-end. (“Modem” = *modulator-demodulator*.) **DOCSIS 3.1 Full Duplex** targets symmetric rates up to about **10 Gbps**.
- **FTTH (Fiber to the Home)**, the highest-capacity consumer option.
- **Leased lines** such as a **T3 (~45 Mbps)** up to roughly **10 Gbps** for businesses.
- In developing regions, **wireless/mobile** access predominates.

The access ISP has a **POP (Point of Presence)** where customer traffic enters its fully digital, packet-switched backbone. ISPs interconnect at **IXPs (Internet eXchange Points)** through **peering** — a business arrangement to exchange traffic. A large IXP such as **AMS-IX (Amsterdam)** connects **800+ ISPs** and carries on the order of **4000 Gbps (4 Tbps)**. Peering relationships come in two flavors: an ISP may **pay another for transit** (carrying its traffic to the rest of the Internet), or two ISPs may agree to **settlement-free peering** (exchanging each other’s traffic for free when it is mutually beneficial). Competitors thus “compete in public and cooperate in private” (Metz 2001). Because routing follows these business relationships, the path a packet takes is **not necessarily the shortest** one. At the top are a handful of **tier-1 ISPs** (e.g., AT&T, Level 3) that pay no one for transit and together form the Internet’s core.

Content lives in data centers: a **server farm** of many machines, often in **colocation/hosting** facilities where companies rent space, or increasingly in virtualized form where a customer rents a **virtual machine**. Because electricity is a dominant cost, data centers are sited near cheap power — e.g., Google’s roughly **\$2 billion** facility at **The Dalles, Oregon**, near hydroelectric power. The classic picture of the Internet as a strict **hierarchy** (home → access ISP → regional → backbone → regional → access ISP → destination) has **flattened**: large content providers and CDNs — the “**hyper-giants**” (Google, Netflix, Twitch, Amazon, and CDNs such as Akamai, Limelight, Cloudflare) — now connect **directly** to access ISPs and even place their own servers *inside* access-ISP networks, bypassing the old backbone hierarchy for most traffic.

1.4.2 Mobile Networks

Mobile (cellular) networks are now the primary way most of the world’s people reach the Internet: there are over **5 billion** mobile subscribers, roughly **65%** of the world’s population, and mobile accounts for more than half of global online traffic.

Architecture

The cellular architecture is quite different from the Internet’s. The authors use **4G LTE** as the reference. The radio side is the **E-UTRAN**, whose base station is the **eNodeB**; the air interface uses **CDMA**-family techniques (covered in Chapter 2). A **radio access network** consists of base stations plus a controller (historically the **RNC, Radio Network Controller**). Behind it is the **core network**: in 4G the all-IP, packet-switched **EPC (Evolved Packet Core)**. (The 3G **UMTS** system evolved from 2G **GSM**; by 4G the network is purely digital — analog is “dead as the dodo.”) Data services trace back to **GPRS**; voice, nominally 64 kbps, is carried compressed at 3–4× less. Within the EPC, the **S-GW (Serving Gateway)** and **P-GW (PDN Gateway)** connect the mobile to external packet networks, with IP carried all the way to the device; **QoS** mechanisms (Chapter 5) give voice and video the timeliness they need.

Two features distinguish mobile networks:

- **Mobility management.** As a user moves, the connection must pass from one base station to the next — a **handover (handoff)**. A **soft handover** (characteristic of CDMA) connects to the new base station *before* releasing the old one; a **hard handover** releases first, then connects. The **HSS (Home Subscriber Server)** tracks each subscriber’s location and handles authentication.
- **Security and billing.** Because carriers bill for usage and must prevent fraud, mobile networks were designed with security from the start. The **SIM card (Subscriber Identity Module)**, introduced with **2G GSM**, is a removable smartcard that both identifies the subscriber and holds cryptographic keys; it can be moved between handsets, is the basis of network security, and makes roaming (or swapping to a new carrier) a matter of changing the SIM. In **UMTS** the phone also verifies that the *network* is legitimate (mutual authentication), and keys on the SIM encrypt traffic over the air — far better than easily-tapped **1G**, though not without weaknesses.

Packet Switching vs. Circuit Switching

A central conceptual contrast — and historically a “war” between two communities — is **packet switching** versus **circuit switching**:

- **Packet switching** is **connectionless**. Every packet is routed independently; if a router fails, the network reconfigures around it with little harm. The downside is that congestion (an oversubscribed link) forces the network to **buffer, delay, or drop** packets, so quality of service is harder to guarantee. This is the model favored by the **Internet (data) community**.
- **Circuit switching** is **connection-oriented**. Before data flows, the network performs **connection setup**, reserving a route that is held for the duration of the call; all data follows that same route, and resources (bandwidth, buffers) are reserved. If a line or switch on the path fails, the call is dropped — so it is **less fault-tolerant** — but because resources are reserved, **QoS is strong** (and if resources are unavailable the call is simply rejected, i.e., you get a busy signal). This is the model favored historically by the **telephone companies**.

In today’s mobile networks both appear: the packet-switched core (EPC) handles data, while legacy voice used a circuit-switched core with elements such as the **MSC (Mobile Switching Center)**, **GMSC (Gateway MSC)**, and **MGW (Media Gateway)** to interwork with the **PSTN (Public Switched Telephone Network)**.

Earlier Generations: 1G, 2G, 3G

1G was **analog** voice; the U.S. system **AMPS (Advanced Mobile Phone System)** launched in **1982**. **2G** went **digital**, gaining capacity, security, and text messaging; **GSM** arrived in **1991**. **3G**, launched around **2001**, provided digital voice *and* broadband data, and brought a thicket of jargon and standards. The **ITU** defined 3G as offering at least **2 Mbps** for stationary or walking users and **384 kbps** in a moving vehicle; the main 3G system, **UMTS**, eventually offered up to about **14 Mbps** downlink and **6 Mbps** uplink. The scarce resource in all of this is **radio spectrum**, which governments sell at **spectrum auctions** — the UK’s 3G auction in **2000** raised roughly **\$40 billion** for five licenses. The core design idea is the **cellular** layout: the coverage area is divided into **cells**, each using a set of frequencies, with **frequency reuse** so that non-adjacent cells can reuse the same frequencies without interfering. 1G required careful frequency planning (reusing a frequency only once every several cells); modern 3G systems tolerate more interference and can reuse all frequencies, often with **sectorized antennas**.

Modern Generations: 4G and 5G

In 4G and 5G, mobile *broadband data* outweighs voice. **4G LTE** (late 2000s) was faster and effectively **outpaced WiMAX (802.16)** as the mobile broadband choice. **5G** (early 2020s) targets up to about **10 Gbps**. A key technical difference is the spectrum used: 4G uses channels up to about **20 MHz**, while 5G uses frequencies up to about **6 GHz** (and higher “millimeter-wave” bands). Higher frequencies give more bandwidth but **shorter range and more attenuation** (microwaves are absorbed by water and rain), which 5G compensates for with many antennas and **MIMO (Multiple-Input Multiple-Output)**.

1.4.3 Wireless Networks (WiFi)

Laptops and phones connect to the Internet “as if by magic” over WiFi. In the mid-1990s, incompatible wireless-LAN products pushed an **IEEE** committee to standardize; the result was named **802.11** (following the earlier 802.1–802.10 groups) and nicknamed **WiFi**.

A defining choice — the **opposite** of cellular — is that WiFi uses **unlicensed ISM (Industrial, Scientific, Medical) bands**, which anyone may use without a license, in exchange for **limited transmit power** (ITU-R designates bands such as **902–928 MHz**, **2.4–2.5 GHz**, and **5.725–5.825 GHz**). The cost of being unlicensed is interference from other devices in the same bands — cordless phones, garage-door openers, microwave ovens, and so on.

WiFi has two modes: devices can talk through an **Access Point (AP / base station)** — the usual infrastructure mode — or directly to one another in an **ad hoc network**. A core physical problem is **multipath fading**: radio signals reflect off objects and arrive via multiple paths, and the copies can reinforce or cancel one another, so signal strength is sensitive to position. The remedy is **path diversity** — sending over multiple independent paths in frequency, space (multiple antennas), or time — so that it is unlikely all copies fade at once.

The 802.11 family evolved rapidly:

- **1997**: the original 802.11, 1–2 Mbps, using frequency hopping and spread spectrum.
- **802.11b (1999)**: **11 Mbps**.
- **802.11a (1999)** and **802.11g (2003)**: up to **54 Mbps** using **OFDM (Orthogonal Frequency Division Multiplexing)**.
- **802.11ac**: up to about **3.5 Gbps**.
- **802.11ad**: up to about **7 Gbps**, but only indoors within a single room (very high frequency, short range).

For sharing the air, WiFi uses **CSMA (Carrier Sense Multiple Access)** — listen before you transmit, and if the channel is busy, defer; this lineage runs from Ethernet back to the **ALOHA** system in Hawaii. After a collision, stations wait a random (and, on repeated collisions, longer) time. A subtle problem is the **hidden-terminal** situation: station A and station C can both reach B but cannot hear each other, so they collide at B without realizing it. When several APs are joined by a **distribution system** (typically a switched Ethernet), a moving device **re-associates** from one AP to another; WiFi mobility is usually **nomadic** (you move, settle, then use the network) rather than **on-the-go** (continuously connected while moving), though **802.21** addresses handover between wired and wireless. Security began with **WEP (Wired Equivalent Privacy)**, which was fatally flawed and broken (Borisov et al., 2001); it was replaced by **WPA/WPA2** from the **802.11i** standard, with **802.1X** providing certificate-based authentication. WiFi is now pervasive on trains, planes, boats, and in cars, and **LTE-U (LTE-Unlicensed)** brings cellular into the same unlicensed bands. WiFi is treated in depth in Chapter 4.

1.5 Network Protocols

A **protocol** is an agreement between the communicating parties on how communication is to proceed. This section covers design goals, the central idea of layering, the connection/reliability dimensions of service, the service primitives, and how services relate to protocols.

1.5.1 Design Goals

Networks are engineered toward several goals, the chief of which are reliability, resource allocation, evolvability, and security.

Reliability

The network must work **correctly even though it is built from unreliable components**. Bits can be corrupted by noise (especially on wireless links) or by hardware and software faults. Two complementary mechanisms address this: **error detection** (codes that let the receiver detect a corrupted message and request retransmission) and **error correction** (codes carrying enough redundancy that the receiver can reconstruct the original without retransmission). Both are used, at low layers (per link) and at high layers (end to end). Reliability also means **finding a working path** through a network even when some links or routers are down — the job of **routing** (e.g., getting a message from Berlin to Paris even if the direct link is broken, by going another way).

Resource Allocation

Networks must share finite resources (link bandwidth, router buffers) among competing users, and must **scale**. The key technique is **statistical multiplexing** — sharing capacity based on the statistics of demand rather than reserving a fixed slice for each user. Two problems arise: **flow control**, preventing a fast sender from overwhelming a slow receiver (handled with feedback from receiver to sender), and **congestion**, where too many senders collectively oversubscribe the network (handled by getting senders to reduce their load). When some applications (e.g., live video) need timely delivery, the network provides **QoS (Quality of Service)** to reconcile real-time demands with raw throughput.

Evolvability

Because networks grow and change over decades, the architecture must accommodate new technologies and designs. The master technique is **protocol layering** (below). Supporting mechanisms include **addressing and naming** (identifying senders and receivers), handling **message ordering** (e.g., sequence numbers so reordered pieces can be put back in order), coping with a maximum message size by **disassembling and reassembling** large messages, and **internetworking** disparate networks.

Security

The network must defend against adversaries. **Confidentiality** protects against eavesdropping (keeping messages secret). **Authentication** prevents impersonation —

making sure you are really talking to your bank and not a spoofed site, or that a cellular user is who they claim. **Integrity** prevents surreptitious tampering, such as an attacker changing “transfer \$10” into “transfer \$1000.” The unifying toolkit is **cryptography**, developed in Chapter 8.

1.5.2 Protocol Layering

To manage complexity, networks are organized as a **stack of layers (or levels)**, each built on the one below. Each layer **offers services** to the layer above it while **hiding the details** of how those services are implemented — a form of information hiding, abstraction, and encapsulation familiar from object-oriented programming; each layer is like a **virtual machine** offering certain operations.

A **layer-*n* protocol** is the set of rules and conventions governing the conversation between the **layer-*n* peers** on two different machines (peers may be software processes, hardware devices, or even people). Crucially, **no data is transferred directly from layer *n* on one machine to layer *n* on another**. Instead, each layer passes data and control to the layer below it, until the bottom (the **physical medium**) is reached and the bits actually cross; on the far side they travel back up. The communication *between peers* is therefore **virtual** (drawn as a dashed horizontal line), while the only *physical* communication is down one stack, across the medium, and up the other (solid lines).

Between each pair of adjacent layers is an **interface** defining the operations and services the lower layer offers the upper one. Good design calls for **clean, well-defined interfaces** that pass the **minimum necessary information**, so that one layer’s implementation can be replaced entirely (e.g., swapping telephone lines for satellite links) without disturbing its neighbors, as long as it still offers the same service. A **network architecture** is the complete set of layers and protocols; a **protocol stack** is the list of the specific protocols a given system uses, one per layer. Importantly, a network architecture’s specification must contain **enough information to implement each layer correctly**, but the **implementation details and the interfaces are *not* part of the architecture**, because they are hidden inside the machines.

The authors illustrate layering with a **philosopher-translator-secretary** analogy. Two philosophers (the layer-3 peers) want to communicate but speak different languages; one thinks “I like rabbits.” Each has a translator (layer 2) who agrees on an intermediate language — say the philosopher’s message is rendered into **Dutch** (“Ik vind konijnen leuk”) — and each has a secretary (layer 1) who agrees on a transmission method, such as fax. The key properties: each layer’s protocol is **independent** of the others (the translators can switch to another common language without the philosophers or secretaries knowing; the secretaries can switch from fax to email without involving the translators), and each layer may **add information** intended only for its peer.

Technically, this added information takes the form of **headers** (and sometimes **trailers**). A message **M** produced at an upper layer (say layer 5) is handed down; the next layer may prepend a header **H4** and pass it on; a lower layer may **split** the message into pieces (M1, M2), each getting its own header (H3), and a link layer may add both a **header and a**

trailer to each piece before transmitting. On the receiving side, each layer **strips off and acts on its own header/trailer** and passes the remainder up, so that each layer's software is written as though communication were horizontal (conceptually calling routines like *SendToOtherSide* and *GetFromOtherSide*). The lowest layers are often implemented in **hardware or firmware**.

1.5.3 Connections and Reliability

A layer offers two broad kinds of service to the layer above — **connection-oriented** and **connectionless** — and each can be **reliable** or **unreliable**.

Connection-Oriented Service

Modeled on the **telephone** system: the user **establishes** a connection, **uses** it, and then **releases** it. The connection acts like a **tube** — bits go in one end and come out the other **in order**. During setup the two sides can **negotiate** parameters such as maximum message size and required quality of service. A connection with reserved resources (such as guaranteed bandwidth) is a **circuit**.

Connectionless Service

Modeled on the **postal** system: each message (a **packet** at the network layer) carries a **full destination address** and is routed **independently** of the others. Networks typically use **store-and-forward switching** (a switch fully receives a packet before forwarding it) or sometimes **cut-through switching** (begin forwarding before the whole packet arrives). Usually the first packet sent arrives first, but because packets are independent they **can be reordered**. Unreliable (unacknowledged) connectionless service is called **datagram** service, by analogy with telegram service.

Reliability

A **reliable** service never loses data: the receiver **acknowledges** each message so the sender knows it arrived, at the cost of extra overhead and delay. (File transfer is a case where reliability is essential — every byte must arrive.) Reliable connection-oriented service comes in two variants: **message sequences**, which preserve message boundaries (sending two 1024-byte messages is distinguishable from one 2048-byte message — like discrete pages of a book), and **byte streams**, which do not (the receiver cannot tell how the stream was chunked — like the frames of a movie running together).

Not everything should be reliable. Reliable service may be unavailable at a given layer (e.g., **Ethernet** provides unreliable delivery and relies on higher layers to recover), and the **delays** introduced by acknowledgment and retransmission are unacceptable for **real-time** traffic. In **VoIP**, for instance, a little noise is tolerable but delay is not; in streaming video, an occasional wrong pixel is far better than a jerky, stalled picture. An intermediate option is the **acknowledged datagram** (like a registered letter with a return receipt), used by services such as text messaging. Another is the **request-reply** service, in which the sender transmits a single request and gets back a single reply (e.g., asking a map server for the Chinese restaurants in a neighborhood). The six combinations are summarized as:

Service	Example
Reliable connection-oriented — message sequence	Sequence of pages of a document
Reliable connection-oriented — byte stream	Downloading a movie / remote login
Unreliable connection-oriented	(rare)
Unreliable connectionless — datagram	Electronic junk mail / broadcast
Acknowledged datagram	Text messaging
Request-reply	Database query

1.5.4 Service Primitives

A service is formally defined by a set of **primitives** (operations) that a user can invoke to get the layer below to perform an action or report on one. In an operating system these are often **system calls** that **trap into the kernel**. The exact set depends on the service. For a simple connection-oriented service, the authors give six primitives:

Primitive	Meaning
LISTEN	Block, waiting for an incoming connection
CONNECT	Establish a connection with a waiting peer
ACCEPT	Accept an incoming connection from a peer
RECEIVE	Block, waiting for an incoming message
SEND	Send a message to the peer
DISCONNECT	Terminate the connection

These correspond (in simplified form) to the **Berkeley socket** interface. The authors walk through a **request-reply** interaction built from them, using packets on the wire: (1) the client's **CONNECT** sends a *connection-request* packet; (2) the server, which was blocked in **LISTEN**, issues **ACCEPT** and a *connection-accepted* packet comes back; (3) the client **SENDS** a *request* and does a **RECEIVE**; (4) the server **RECEIVES** it, processes it, and **SENDS** the *reply*; (5) the client receives the reply and issues **DISCONNECT**, sending a *disconnect* packet; (6) the server **DISCONNECTS** in return. (A real sequence like this uses on the order of six packets.) **LISTEN** and **RECEIVE** are **blocking** calls; **CONNECT** must specify whom to connect to. An analogy: the server is like a customer-service manager whose phone (**LISTEN**) waits for calls.

Why not just use a simpler connectionless scheme? Because with large messages that must be split, packets can be **lost, reordered, or arrive with pieces missing**, and the receiver would have trouble telling when the last packet of a message had arrived or distinguishing packet 1 of one file from packet 1 of another. The connection machinery handles all this. Primitives and connection management are developed fully in Chapter 3.

1.5.5 The Relationship of Services to Protocols

Services and protocols are **distinct concepts**, and conflating them is a classic design error.

A **service** is the set of primitives (operations) that a layer provides to the layer **above** it. The service defines **what** operations are available — not **how** they are implemented — and says nothing about how the operations are realized. It sits at the **interface** between two layers, where the lower layer is the **service provider** and the upper layer is the **service user**.

A **protocol**, by contrast, is the set of **rules governing the format and meaning of the packets (or messages) exchanged by the peer entities *within* a layer**. Entities use protocols to implement their service definitions. The payoff of keeping them separate is that the entities are free to **change the protocol at will, as long as they keep offering the same service** to their users — the two are decoupled, exactly like an abstract data type or object whose operations (service) are separate from its implementation (protocol). The authors note that many older protocols did *not* make this distinction (for example, a layer might expose a raw “SEND PACKET” operation), so any protocol change was visible to the user above — which they call a serious design blunder.

1.6 Reference Models

A layered design needs a clear statement of what each layer does and how layers interact — a **reference model**. Two have been historically important: the **OSI** model and the **TCP/IP** model. This book ultimately uses a **hybrid** of the two.

1.6.1 The OSI Reference Model

The **OSI (Open Systems Interconnection)** model was proposed by **ISO** (Day and Zimmermann, 1983; revised 1995) to standardize how “open” systems — systems open to communicating with others — interconnect. It has **seven layers**, built on **five principles**: (1) create a layer wherever a different level of abstraction is needed; (2) give each layer a well-defined function; (3) choose layer functions with internationally standardized protocols in mind; (4) place boundaries to minimize information flow across interfaces; and (5) use enough layers that distinct functions are not thrown together, but not so many that the architecture becomes unwieldy.

Three concepts are central to OSI — **services, interfaces, and protocols** — and the model’s **single biggest contribution** was making the distinction between them **explicit**: a layer’s **service** is what it does for the layer above; its **interface** tells the upper layer how to access it; and its **peer protocol** is its own internal business, changeable as long as the service is preserved. (This is exactly the service-versus-protocol discipline of §1.5.5, which TCP/IP did not originally articulate.)

The seven layers, from top to bottom, each with the name of the data unit it handles, are:

Layer	Name	Data unit
7	Application	APDU (Application Protocol Data Unit)
6	Presentation	PPDU

5	Session	SPDU
4	Transport	TPDU
3	Network	Packet
2	Data link	Frame
1	Physical	Bit

In outline: the **physical** layer moves raw bits over a medium; the **data link** layer turns a raw facility into a line that appears free of transmission errors and handles access to a shared medium; the **network** layer controls subnet operation, especially routing and congestion; the **transport** layer provides end-to-end data transfer, splitting data from the session layer and ensuring it arrives correctly; the **session** layer manages dialogs (sessions) including synchronization; the **presentation** layer deals with the syntax and semantics of transmitted information (data representation); and the **application** layer contains the protocols users actually need, such as HTTP for the Web.

1.6.2 The TCP/IP Reference Model

The **TCP/IP** model grew from the ARPANET and defines the Internet. Because ARPANET had to interwork leased lines with satellite and radio networks, the overriding design goal became the **seamless connection of multiple networks**; a second goal was **survivability** — the network should keep conversations alive as long as source and destination machines were up, even if intermediate subnet hardware was lost (Cerf and Kahn, 1974; see also Clark 1988 and Braden 1989 on the design philosophy). It also had to be **flexible** enough to support wildly different applications, from file transfer to real-time speech. The model has four layers.

Link Layer

The lowest layer is the **link layer**. It is best described not as a true layer in the usual sense but as an **interface** between hosts and the transmission links, describing what a link (a serial line, classic Ethernet) must do to carry packets. Early accounts barely specified it.

Internet Layer

The **internet layer** is the linchpin that holds the architecture together. Its job is to let a host inject **packets** into any network and have them travel independently to the destination, possibly over multiple networks and possibly arriving **out of order** (higher layers reorder them). It defines the official packet format and protocol, **IP (Internet Protocol)**, together with **ICMP (Internet Control Message Protocol)**. Delivering IP packets is its purpose, and the key issues are **routing** (largely solved) and avoiding **congestion** (needing help from higher layers). The authors liken it to the postal system.

Transport Layer

The **transport layer** lets peer entities on source and destination hosts hold a conversation, as in OSI. It defines two protocols. **TCP (Transmission Control Protocol)** is a **reliable, connection-oriented** protocol delivering a **byte stream** with no errors to any machine on

the Internet; it segments the byte stream, handles acknowledgment and reassembly, and provides **flow control** so a fast sender cannot swamp a slow receiver. **UDP (User Datagram Protocol)** is **unreliable and connectionless**, for applications that do their own sequencing/flow control or that value **promptness over accuracy** — one-shot request-reply interactions and real-time **speech and video**, where timely delivery beats perfect delivery.

Application Layer

TCP/IP has **no session or presentation layers** — experience with OSI showed they were rarely useful, so applications simply include any such functions themselves. The **application layer** sits directly on transport and holds the high-level protocols: early ones were **TELNET** (remote login), **FTP** (file transfer), and **SMTP** (email); later came **DNS** (mapping host names to IP addresses), **HTTP** (the Web), and **RTP** (real-time media).

1.6.3 A Critique of the OSI Model and Protocols

Neither model is perfect. Around 1989, OSI was widely expected to take over the world; it did not. The authors attribute this to four factors — **bad timing, bad technology (design), bad implementations, and bad politics**.

Bad Timing

Timing is explained with David Clark's "**apocalypse of the two elephants**." When a new subject appears, a first "elephant" — a burst of **research activity** — rises and falls; later a second elephant — a wave of **billion-dollar investment** — rises. Standards should be written in the **trough** between the two: too early, and the subject is poorly understood; too late, and so much has been invested in other approaches that the standard is ignored. OSI standards appeared just as **TCP/IP** was already in widespread use in research universities. Vendors, wary of being first, offered no OSI products; with nothing available, people stayed on TCP/IP, and OSI never gained momentum.

Bad Technology (Design)

The **choice of seven layers was more political than technical**. Two layers (session and presentation) are nearly empty, while two others (data link and network) are overfull. The model and protocols are **extraordinarily complex** — the printed standards form a stack of paper roughly a meter high — and hard and inefficient to implement. (Mockapetris quipped, via a riddle retold by Rose in 1993, that mixing a mobster and an international standard gives someone who makes you an offer you can't *understand*.) Several functions — **addressing, flow control, and error control** — reappear in layer after layer; as Saltzer et al. (1984) argued, error control is most effective in the **highest** layer, so repeating it lower down is often inefficient.

Bad Implementations

Early OSI implementations were huge, unwieldy, and slow, and the word "OSI" became associated with poor quality. By contrast, **TCP/IP** shipped as part of **Berkeley UNIX**, was

good, and was essentially **free**, so it spread in an upward spiral of adoption while OSI spiraled down.

Bad Politics

TCP/IP was seen as belonging to the researchers and to UNIX (as American as motherhood and apple pie), whereas OSI was perceived as the creature of European **telecom ministries**, the **European Community**, and the **U.S. government** — bureaucrats, many thought, trying to force a technically inferior standard on working researchers, rather like earlier reactions to IBM's PL/I or the DoD's Ada language.

1.6.4 A Critique of the TCP/IP Reference Model

TCP/IP has its own flaws. It **does not clearly distinguish service, interface, and protocol**, so it is a poor guide for designing *new* networks using different technologies. It is **not general**: it is so tied to TCP/IP that it cannot describe any non-TCP/IP stack (trying to use it to describe **Bluetooth**, for example, is essentially impossible). Its **link layer is not really a layer** at all but an interface, and the model **fails to distinguish the physical layer from the data link layer**, which are entirely different in kind. Finally, many of the earliest protocols were **ad hoc**, produced by graduate students and handed around, becoming entrenched and later hard to replace even when awkward — for instance **TELNET**, designed for 10-characters-per-second mechanical Teletype terminals, still in use half a century later.

1.6.5 The Model Used in This Book

The book takes OSI's **model** (its strength) and TCP/IP's **protocols** (their strength), discarding OSI's presentation and session layers and TCP/IP's conflation of physical and link. The resulting **hybrid model has five layers**:

Layer	Role
5 — Application	Programs that use the network: HTTP and other application protocols; DNS and similar “glue”
4 — Transport	Strengthen the network layer's delivery guarantees to what applications want — typically a reliable byte stream (TCP)
3 — Network	Combine multiple links into networks, and networks of networks (internetworks), and find a path across them (IP)
2 — Link	Move finite-length messages between directly connected computers with a given level of reliability (Ethernet, 802.11)
1 — Physical	Transmit bits across a medium as signals (voltages, light, radio)

The **physical** layer concerns sending raw bits as signals over a channel; the **link** layer moves discrete, finite messages between *directly connected* machines with some reliability; the **network** layer combines links into networks and internetworks and routes across them; the **transport** layer upgrades the delivery guarantees to what applications need (e.g., a reliable byte stream); and the **application** layer holds the programs that put the network to use, with DNS and HTTP as examples. The rest of the book is organized bottom-up along exactly these five layers.

1.7 Standardization

Innovation in networking depends not only on technology but on **standards** (and increasingly on **policy and law**, the subject of §1.8). Standardization lets equipment from many vendors interoperate and enlarges the market, enabling mass production and **economies of scale**.

1.7.1 Standardization and Open Source

Without coordination among the many vendors, there would be chaos; standards fix that and grow the market. The art is deciding **what belongs in a standard**: just enough to guarantee **interoperability**, and no more. For example, **802.11** standardizes the transmission rates but deliberately does *not* dictate when a device should use which rate — that is left to implementers. Industry groups such as the **WiFi Alliance** test and certify interoperability, and the **ONF (Open Networking Foundation)** produces both standards and **open-source** software for SDN. Note that a **protocol** standard specifies what goes “over the wire,” *not* the service interface or API; TCP and IP are standardized while the socket API is a widely used but separate convention — good APIs (like **Berkeley sockets**) are valuable in their own right.

Standards are of two kinds. **De facto** (“from the fact”) standards arise from practice without any formal plan — examples include **HTTP** (which began as practice) and **Bluetooth** (developed by Ericsson). **De jure** (“by law”) standards are adopted by a formal, recognized standards body. Standards bodies come in two sorts: those established by **international treaty** among governments, and those that are **voluntary, non-treaty** organizations. Often a de facto standard is later blessed as de jure (HTTP was eventually standardized through the IETF), and bodies frequently ratify one another’s work. Business **alliances** also set standards — e.g., **3GPP (Third Generation Partnership Project)** standardized UMTS/3G. The major players are the **ITU, ISO, IETF, and IEEE**.

1.7.2 Who’s Who in the Telecommunications World

Telephony’s legal status differs by country. The U.S. historically had many private companies; **AT&T** — once the world’s largest company, providing phone service to about **80%** of U.S. phones — was broken up in **1984**, and the **Telecommunications Act of 1996** opened the field to competition (though the hoped-for competition gave way to later consolidation). At the opposite extreme, many countries ran telephony as a **government**

monopoly called a **PTT (Post, Telegraph & Telephone)** administration; the worldwide trend has been **liberalization** and privatization.

The key treaty body is the **ITU (International Telecommunication Union)**, whose predecessor was founded in **1865** to standardize international telegraphy and which became a **UN agency in 1947**. Its members include about **200 governments** (the U.S. is represented by the State Department, since it has no PTT) plus over **700 sector and associate members** — carriers (AT&T, Vodafone, Sprint), equipment makers (Cisco, Nokia, Nortel), computer firms (Microsoft, Dell, Toshiba), chip makers (Intel, Motorola, TI), and others (Boeing, CBS, VeriSign). The ITU has three sectors: **ITU-T** (Telecommunication Standardization — formerly **CCITT** until 1993), **ITU-R** (Radiocommunications — allocates **radio spectrum**), and **ITU-D** (Development — addresses the digital divide). ITU-T output takes the form of **Recommendations** (governments may adopt or ignore them). Work is organized into **Study Groups** (which can involve up to 400 people — e.g., SG 15 handles fiber to the home), subdivided into Working Parties and Expert Teams; the ITU has issued thousands of Recommendations, including **H.264** (video coding, also known as MPEG-4 AVC) and **X.509** (public-key certificates) (Irmer, 1994).

1.7.3 Who's Who in the International Standards World

The voluntary, non-treaty body is **ISO (International Standards Organization)**, founded in **1946**, whose members are the national standards bodies of **161 countries** — including **ANSI** (U.S.), **BSI** (Britain), **AFNOR** (France), and **DIN** (Germany). ISO issues standards on a vast range of subjects (famously, ISO 2451 for cocoa beans, ISO 1530 for fishing nets, ISO 4416 for women's underwear), cooperates with ITU-T, and has published over **21,000** standards, including the OSI standards. Its structure is **TCs (Technical Committees)** — more than 200 of them (TC1 covers screw threads) — subdivided into **SCs (subcommittees)** and **WGs (working groups)**, drawing on over **100,000** volunteers. For information technology, ISO and the **IEC** created the joint committee **JTC1** in 1987 (merging the former TC97). The approval procedure moves a document from **CD (Committee Draft)** — circulated six months for criticism — to **DIS (Draft International Standard)** — for comments and voting — to final **IS (International Standard)**, a process that takes years.

In the U.S., **NIST (National Institute of Standards and Technology)**, part of the Department of Commerce and formerly the National Bureau of Standards, issues standards that are mandatory for government purchases (except the DoD). The **IEEE (Institute of Electrical and Electronics Engineers)**, the world's largest professional organization, runs the **802** committee that standardizes LANs. Its many working groups have had a low overall success rate, but two — **802.3 (Ethernet)** and **802.11 (wireless LANs)** — have been enormously successful. Notable 802 groups include:

Number	Topic	Status
802.1	Overview and architecture of LANs	—
802.2	Logical link control	(Hibernating)

802.3 *	Ethernet	Dominant
802.4	Token bus	(Defunct)
802.5	Token ring (IBM)	(Faded)
802.11 *	Wireless LANs (WiFi)	Dominant
802.15	Personal area networks (Bluetooth, ZigBee)	—
802.16	Wireless MAN (WiMAX)	(Faded)

(An asterisk marks the two big successes; a number of other groups quietly gave up.)

1.7.4 Who's Who in the Internet Standards World

The Internet's standards culture is deliberately different — “suits versus jeans.” Where ITU and ISO meetings are full of corporate executives and civil servants in suits, early Internet meetings resembled a gathering in shorts and T-shirts (many held in San Diego). As the network grew, an informal committee gave way, in **1983**, to the **IAB (Internet Activities Board)** — “herding cats” — later renamed the **Internet Architecture Board**. Its handful of members oversaw task forces and published decisions as **RFCs (Request For Comments)**, archived at www.ietf.org/rfc, numbered chronologically, now numbering over **8000**.

By **1989** the Internet had grown too large for this structure, so in that summer the IAB was reorganized into two bodies, both under it: the **IRTF (Internet Research Task Force)** for long-term research and the **IETF (Internet Engineering Task Force)** for shorter-term engineering problems. The **Internet Society** was created (modeled on professional societies like the ACM and IEEE) with elected trustees who appoint the IAB's members. The IETF is organized into **working groups** (more than 70), grouped into **areas** whose chairs sit on a steering committee.

The Internet's own formal standards track moves a document from **Proposed Standard** (an RFC with enough interest) to **Draft Standard** (requiring working implementations tested at two independent sites for at least four months) to full **Internet Standard**; some eventually become DoD **MIL-STDs**. Finally, the **W3C (World Wide Web Consortium)**, founded by **Tim Berners-Lee in 1994**, develops Web standards; it has roughly 500 members and has issued over 100 **W3C Recommendations** on HTML, privacy, and related topics.

1.8 Policy, Legal, and Social Issues

Networks, like the printing press before them, raise difficult public issues: they do enormous good but can also be misused. This section surveys online speech, net neutrality, security, privacy, and disinformation.

1.8.1 Online Speech

Networks let anyone share views with a vast audience, which collides with society's rules about **politics, religion, and sex** and with content that is offensive to some or outright **illegal** (child pornography, incitement to violence). This forces the question of a platform's responsibility to moderate. In the U.S., **Section 230** of the Communications Decency Act granted online platforms broad **immunity** from liability for what their users post, on the theory that they are neutral "platforms" rather than publishers. That theory has been **eroding** as platforms increasingly **curate** and rank content, and the legal pendulum has begun to swing (e.g., laws targeting online sex trafficking). Platforms increasingly rely on **automated, machine-learning classifiers** to flag content at scale, but these produce **false positives**, raising the specter of **automated censorship**. Separately, the recording and film industries use the **DMCA (Digital Millennium Copyright Act)**: a copyright holder sends a **takedown notice**, and an ISP or content provider that promptly passes it on and complies is shielded from liability — putting the onus on the copyright holder to find infringement, with users able to push back against erroneous notices.

1.8.2 Net Neutrality

Because ISPs can **block or prioritize** traffic, **network neutrality** (a term due to **Tim Wu, 2003**) argues they should not discriminate. It is usually expressed as **four rules: no blocking** of lawful content, **no throttling** of lawful traffic, **no paid prioritization** (no "fast lanes" for those who pay), and **transparency** about network-management practices. Neutrality does *not* forbid all prioritization — prioritizing real-time gaming or video over an overnight file backup is **reasonable network management** — but it is meant to prevent **anti-competitive** behavior, such as **AT&T blocking FaceTime**, or an ISP degrading **Netflix** to favor its own video-on-demand. The rules differ by country, and in the U.S. enforcement has swung back and forth with the **FCC's** authority being granted and revoked; the crux is whether broadband is a **common carrier** (a regulated utility) or an **information service** (like Google or Facebook), a classification that is increasingly hard to make. The FCC **abolished** U.S. net-neutrality rules on **June 11, 2018**, after which several **states** adopted their own. A related practice is **zero rating**: a carrier counts most traffic against a data cap but **exempts** one service — e.g., charging for Netflix while letting its own or a partner's service flow free, or Facebook's **Free Basics** offering a limited set of sites at no data cost — which critics say runs afoul of neutrality's spirit.

1.8.3 Security

The Internet's open design spurs innovation but also enables attack; security is the subject of Chapter 8. A prominent threat is the **DDoS (Distributed Denial of Service)** attack, which exhausts a target's resources using a **botnet** — a large collection of compromised machines under an attacker's control. The **IoT** is a dangerous new source of such machines: insecure "smart" devices (the memorable image is smart toasters helping take down a major website), whose manufacturers have little incentive to secure them, so the cost falls on network operators — a problem of **misaligned incentives** that may require regulation. **Spam** makes up well over **90%** of email, sent to address lists harvested by the millions; it is fought with **filtering** that examines either message **contents** or the sender's **IP address**

and sending patterns. Phishing messages masquerade as legitimate (e.g., your bank) to trick users into revealing credentials, enabling **identity theft**.

1.8.4 Privacy

Networks make it very easy to **collect data** about people and to **snoop**. A striking number of parties are positioned to observe a user: the access ISP, the mobile carrier, the apps on the device, the websites visited, cloud providers, CDNs, device manufacturers, advertisers, and dedicated tracking vendors — enabling **profiling and tracking**. Common mechanisms include **cookies** and **browser fingerprinting** (identifying a browser by its configuration). Service providers can read content directly — e.g., Google historically scanned **Gmail** to target ads. **Location** is especially sensitive (Beresford and Stajano, 2003): an OS vendor can know a device’s precise position (even altitude, via the barometric sensor), as can the carrier via cell towers. Users try to hide with **VPNs** and **Tor** (onion routing) to obscure the source of their traffic, but a VPN operator can see unencrypted traffic, and Tor has weaknesses (e.g., when a single entity controls key relays). There are hard **legal** questions too, such as anonymous whistleblowing versus a defendant’s right to confront an accuser, and cross-border data access — illustrated by the **Microsoft** case that reached the U.S. Supreme Court over email stored on servers in **Ireland** — because the network is inherently **borderless**.

1.8.5 Disinformation

The network lets people find information quickly, but much of it is **misleading or simply wrong** — a search on a medical question may surface advice from a Nobel laureate and from a college dropout side by side, with no obvious way to tell them apart. The problem is sharpest for **news and current events**: the **2016 U.S. election** made “fake news” a central concern. Three challenges stand out: **defining** disinformation, **reliably detecting** it, and deciding **what to do** about it once found — all genuinely hard.

1.9 Metric Units

Networking uses **metric** units, not English ones (no furlongs, stones, or fortnights). Prefixes are abbreviated by their first letters, capitalized when the value is ≥ 1 (so **KB**, **MB**); the one notable exception is lowercase **k** in **kbps**. The main prefixes:

Factor	Prefix	Factor	Prefix
10^{-3}	milli (m)	10^3	kilo (K/k)
10^{-6}	micro (μ)	10^6	mega (M)
10^{-9}	nano (n)	10^9	giga (G)
10^{-12}	pico (p)	10^{12}	tera (T)
10^{-15}	femto (f)	10^{15}	peta (P)

10^{-18}	atto (a)	10^{18}	exa (E)
10^{-21}	zepto (z)	10^{21}	zetta (Z)
10^{-24}	yocto (y)	10^{24}	yotta (Y)

A subtlety: since **milli** and **micro** both begin with “m,” milli is written **m** and micro uses the Greek **μ**. The crucial convention used throughout the book: for **data rates**, prefixes are **powers of ten** — 1 Mbps = 10^6 bits/second, 10 Mbps = 10,000,000 bits/second — but for **memory, disk, and file sizes**, prefixes are **powers of two** — 1 KB = 2^{10} = 1024 bytes, 1 MB = 2^{20} bytes, 1 GB = 2^{30} bytes, 1 TB = 2^{40} bytes. Note also that a capital **B** means **bytes** (8 bits) and a lowercase **b** means **bits**. So, to summarize the book’s rule: **KB/MB/GB/TB are powers of two (storage), while kbps/Mbps/Gbps/Tbps are powers of ten (line speeds)** — a frequent source of confusion, especially for disk sizes.

1.10 Outline of the Rest of the Book

The book interleaves **principles** with **practice**, drawing most examples from the **Internet** and from **wireless** networks (as the two dominant cases), and is structured bottom-up along the five-layer hybrid model. **Chapter 2** covers the **physical layer** and the foundations of data communication (the telephone system / PSTN, the mobile phone system, and cable). The **data link layer** is split across two chapters: **Chapter 3** treats sending packets across a single link, including error detection and correction (with DSL as an example), and **Chapter 4** treats the **medium access control sublayer** — how to share a broadcast channel — covering 802.11 wireless, Ethernet, and switches. **Chapter 5** is the **network layer**: routing (static and dynamic algorithms), congestion control, quality of service, internetworking, and the Internet’s own network layer. **Chapter 6** is the **transport layer**: connection management, UDP and TCP, and performance. **Chapter 7** is the **application layer**: DNS, email, the Web, multimedia, and content distribution (CDNs and peer-to-peer). **Chapter 8** covers **network security** and the cryptography behind it. **Chapter 9** provides suggested readings and the bibliography. The authors also maintain companion websites for the book.

1.11 Summary

Computer networks have many uses — access to information, person-to-person communication, electronic commerce, entertainment, and the Internet of Things — for individuals, companies, and society, with accompanying social, legal, and policy concerns. Networks can be classified by **role** (access, content-provider, transit, and enterprise networks) and by **scale**, from **PANs** through **LANs**, **MANs**, and **WANs** up to **internetworks**, of which the **Internet** is the prime example; a key building block is the **router**, a network-layer gateway that ties networks together.

Network software is built on **protocols** — rules by which peer processes communicate — organized into a **stack of layers**, each providing a **service** to the layer above while hiding

its implementation, and it is essential to keep **service, interface, and protocol** distinct. Services are **connection-oriented or connectionless** and **reliable or unreliable**, and are accessed through **service primitives**. Two **reference models** matter: **OSI** has seven layers and an excellent conceptual model but failed in the market, while **TCP/IP** has four layers and weak conceptual separation but dominant, successful protocols; this book uses a **five-layer hybrid** (physical, link, network, transport, application). Networking depends on **standards** set by bodies such as the ITU, ISO, IEEE, and IETF, and on evolving **policy and law** around speech, net neutrality, security, privacy, and disinformation. Finally, the field uses **metric units**, with the important convention that storage sizes use powers of two (KB, MB, GB, TB) while data rates use powers of ten (kbps, Mbps, Gbps, Tbps).